

Introduction to Deep Learning for Natural Language Processing

8. GPT

Pre-training in NLP

자연어 처리

=====

자연어 이해 (NLU) 텍스트 분류, 개체명 인식
자연어 생성 (NLG) 텍스트 생성하는 영역 >> 디코더

=====

seq2seq : 인코더 - 디코더
인코더가 번역하고자 하는 문장 >> 자연어 이해
디코더가 인코더의 정보로부터 >> '자연어 생성'

=====

트랜스 포머 : 인코더-디코더 구조

=====

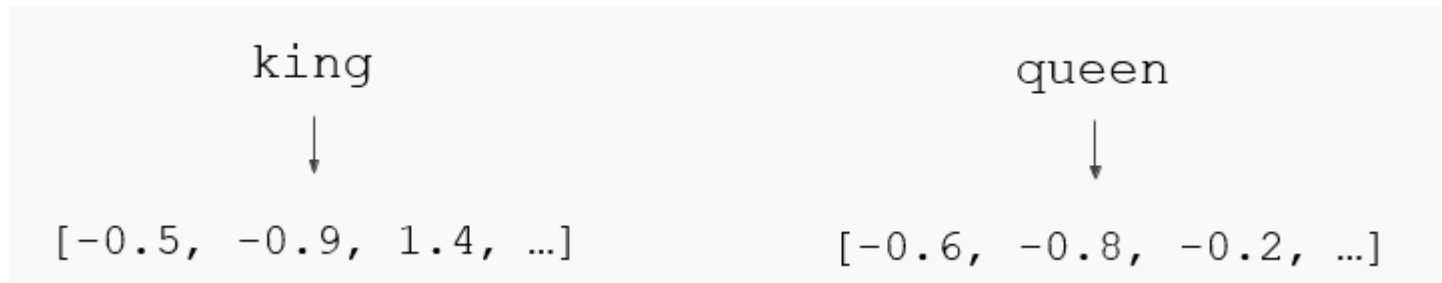
디코더 : RNN 언어 모델, 트랜스포머 디코더 언어 모델
- 이전 단어들로부터 다음 단어를 예측하는 방식
>> 이 방식을 pre-training 에 활용한 것 >> ELMo, GPT

인코더 : 트랜스포머 인코더를 가지고 MASK를 활용, pre-training
>> BERT

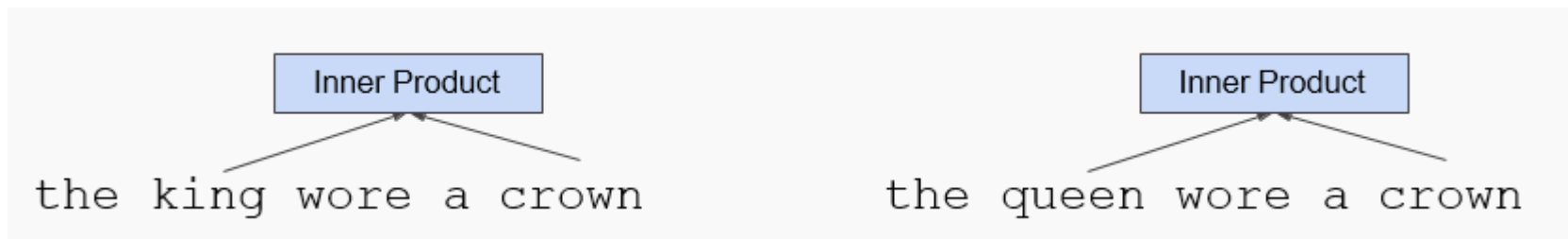
인코더-디코더 상태에서 pre-training : BART, T5

Pre-training in NLP

- 워드 임베딩은 딥 러닝 자연어 처리의 기본.



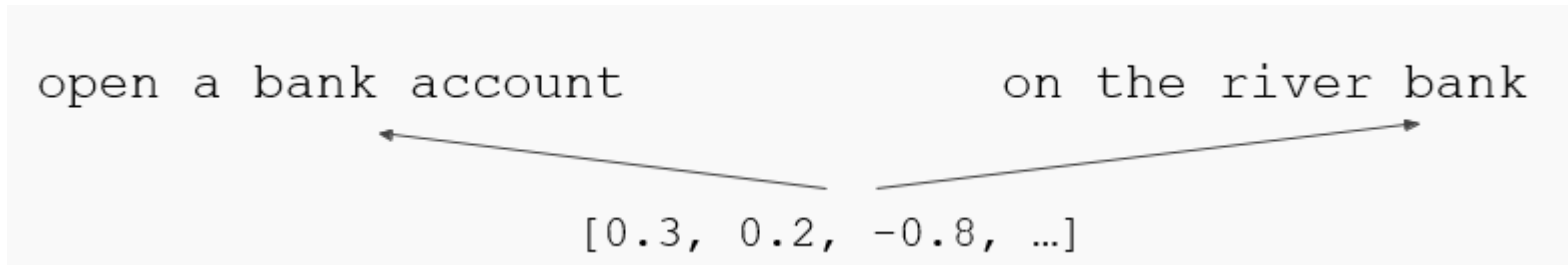
- 사전 훈련된 임베딩(Word2Vec, GloVe)은 대용량 텍스트의 단어들의 동시 등장 통계로부터 훈련시키는 방법.



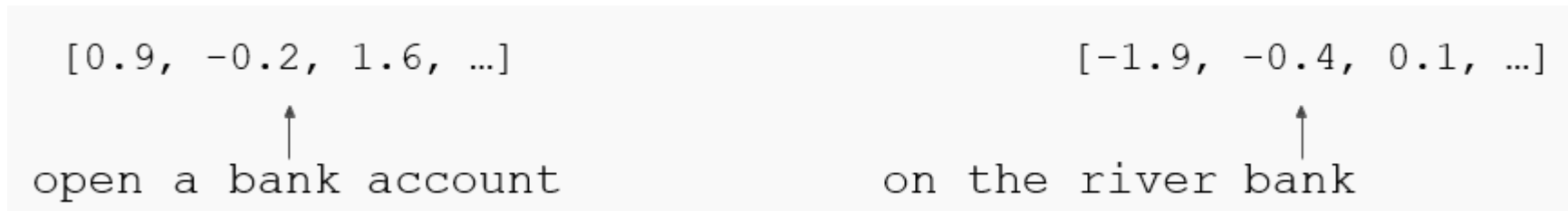
Contextual Representation

- 문제점 : 단어는 문맥에 따라서 뜻이 달라질 수 있음.

Ex) 동음이의어. 배, 사과 등.



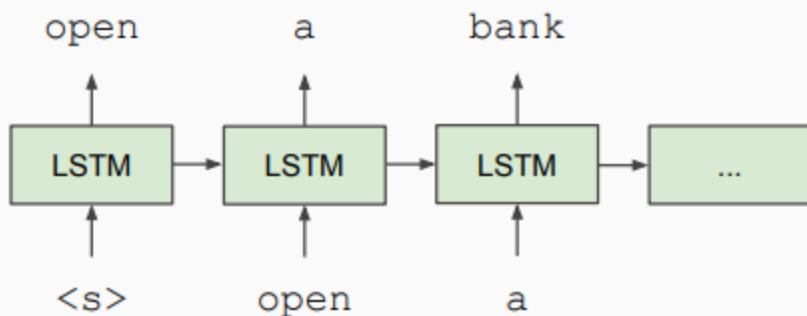
- 해결 방법 : 언어 모델을 사전 훈련 시켜서 문맥에 따른 표현(Contextual Representation)을 얻는다.



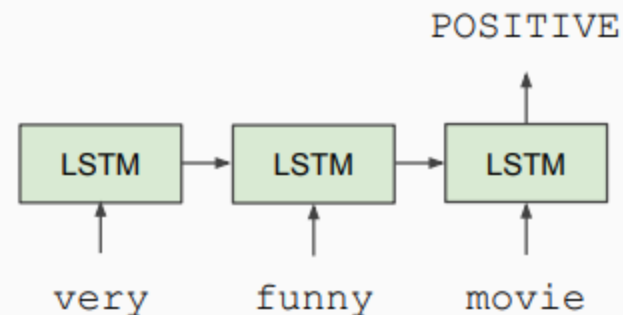
History of Contextual Representations

- *Semi-Supervised Sequence Learning, Google, 2015*

Train LSTM Language Model



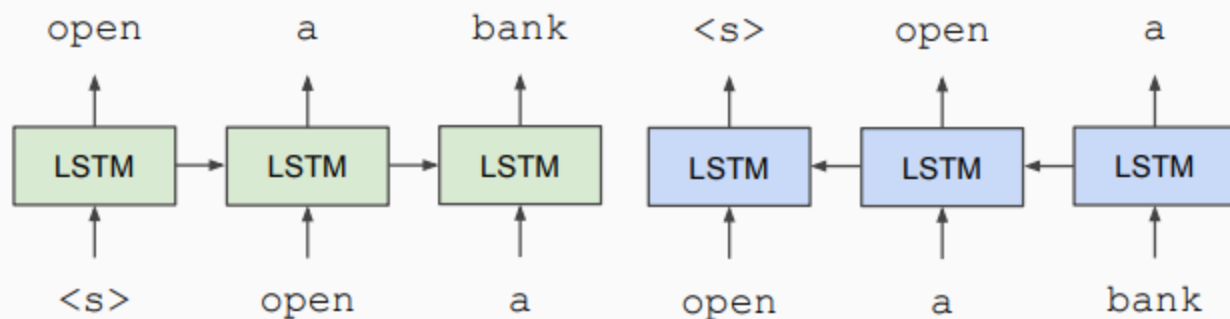
Fine-tune on Classification Task



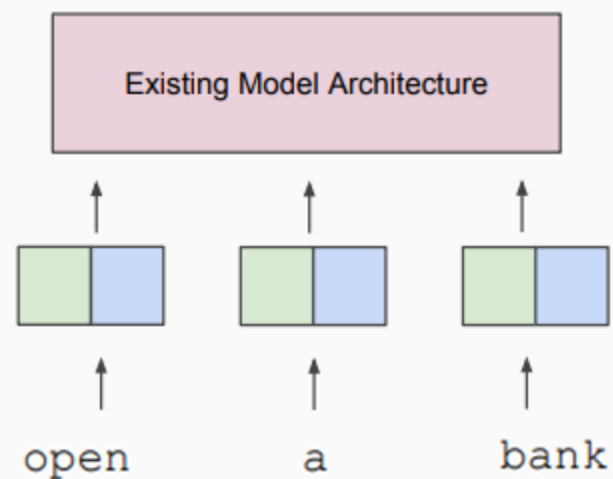
History of Contextual Representations

- *ELMo: Deep Contextual Word Embeddings*, AI2 & University of Washington, 2017

Train Separate Left-to-Right and Right-to-Left LMs



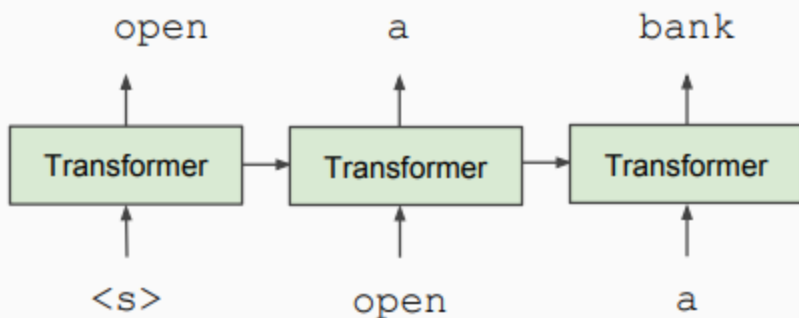
Apply as “Pre-trained Embeddings”



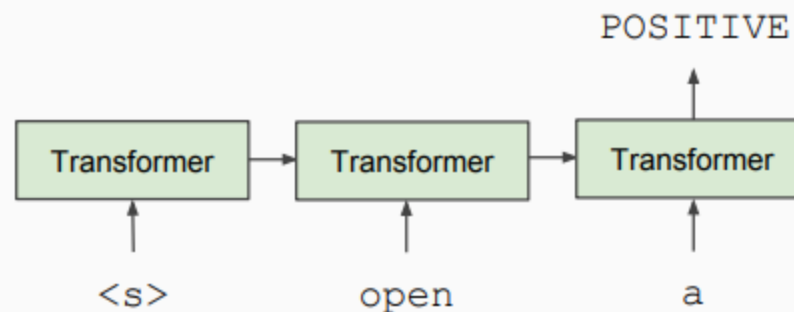
History of Contextual Representations

- *Improving Language Understanding by Generative Pre-Training*, OpenAI, 2018

**Train Deep (12-layer)
Transformer LM**



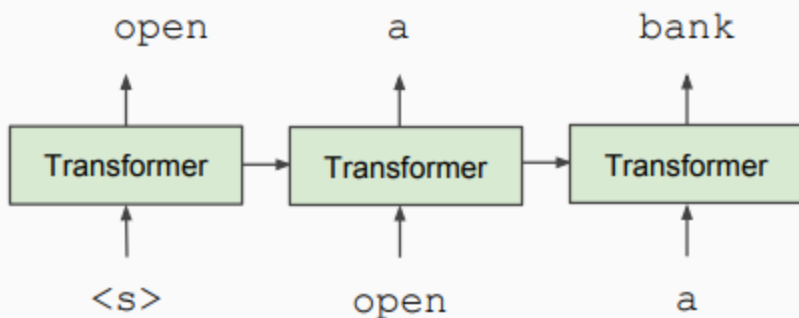
**Fine-tune on
Classification Task**



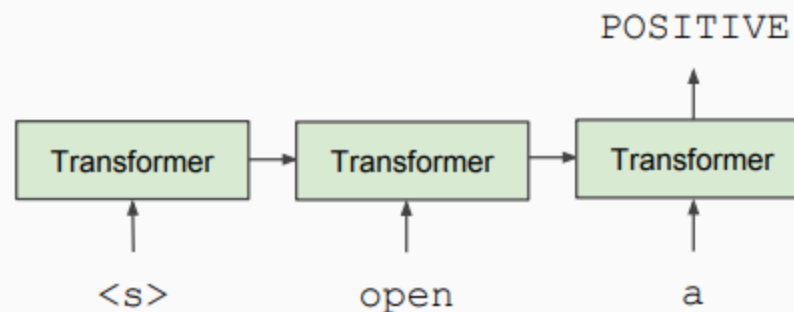
History of Contextual Representations

- *Improving Language Understanding by Generative Pre-Training*, OpenAI, 2018 이 모델을 GPT라고 한다.

**Train Deep (12-layer)
Transformer LM**



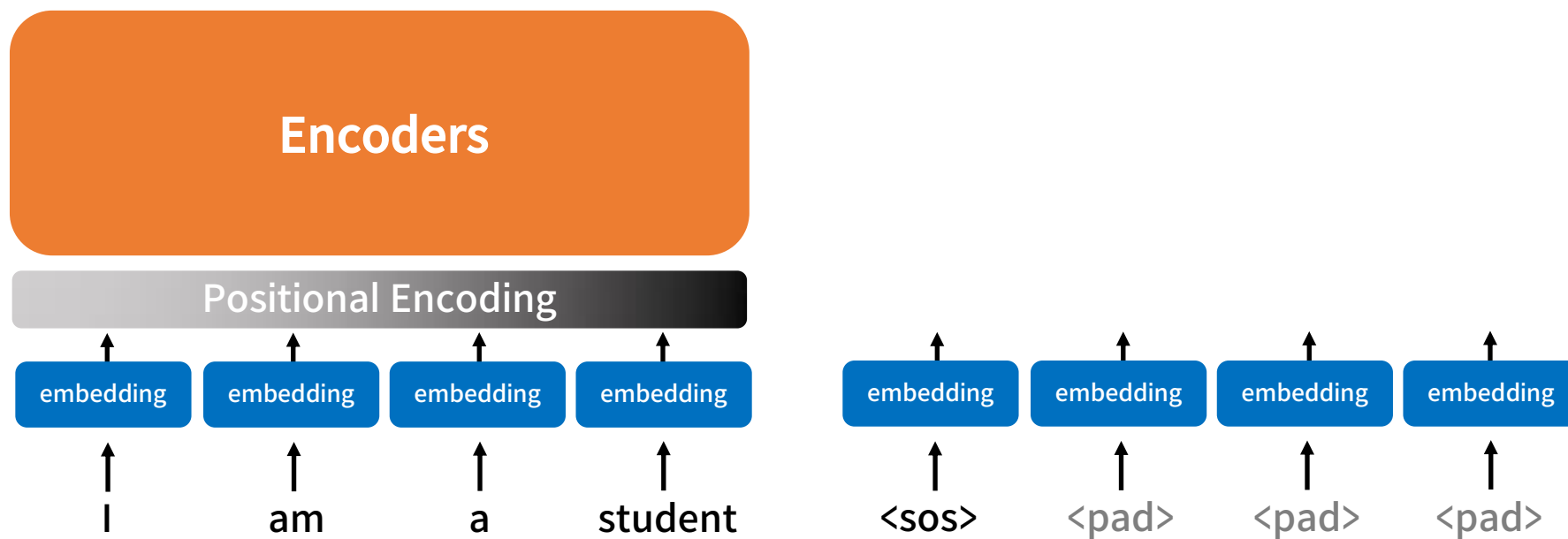
**Fine-tune on
Classification Task**



Generative Pre-trained Transformer

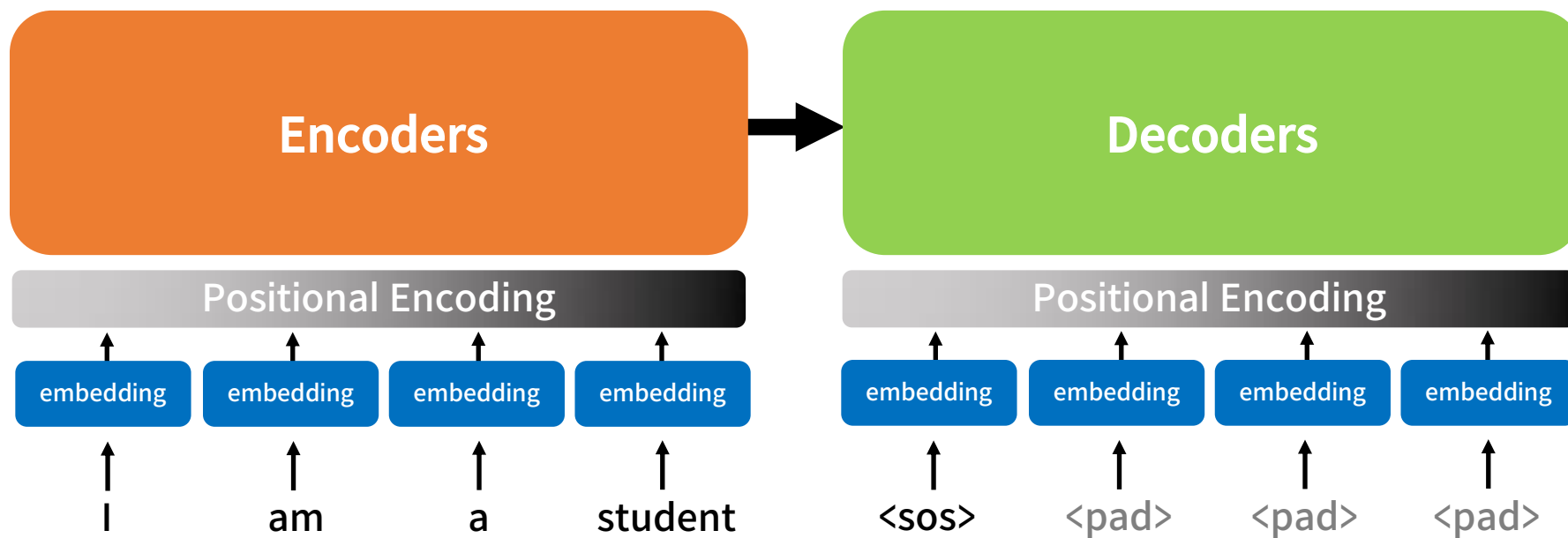
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



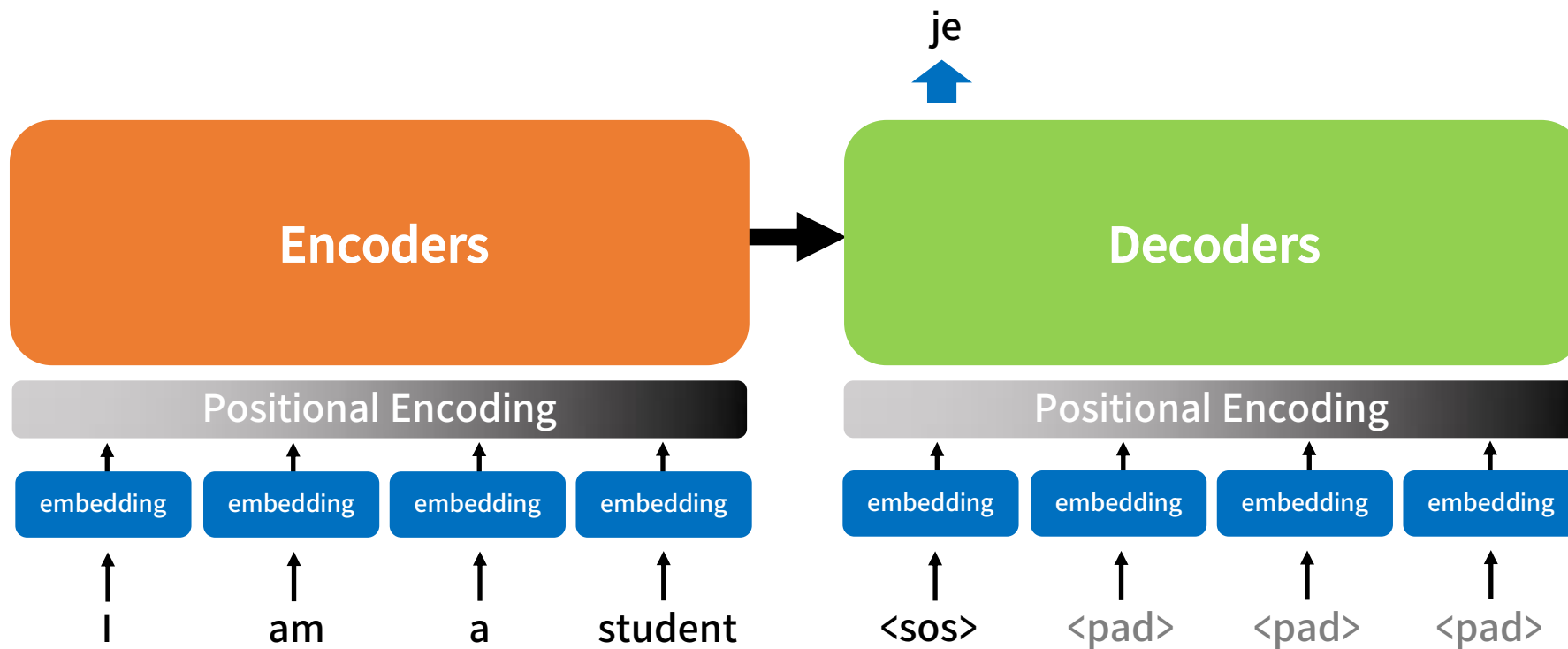
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



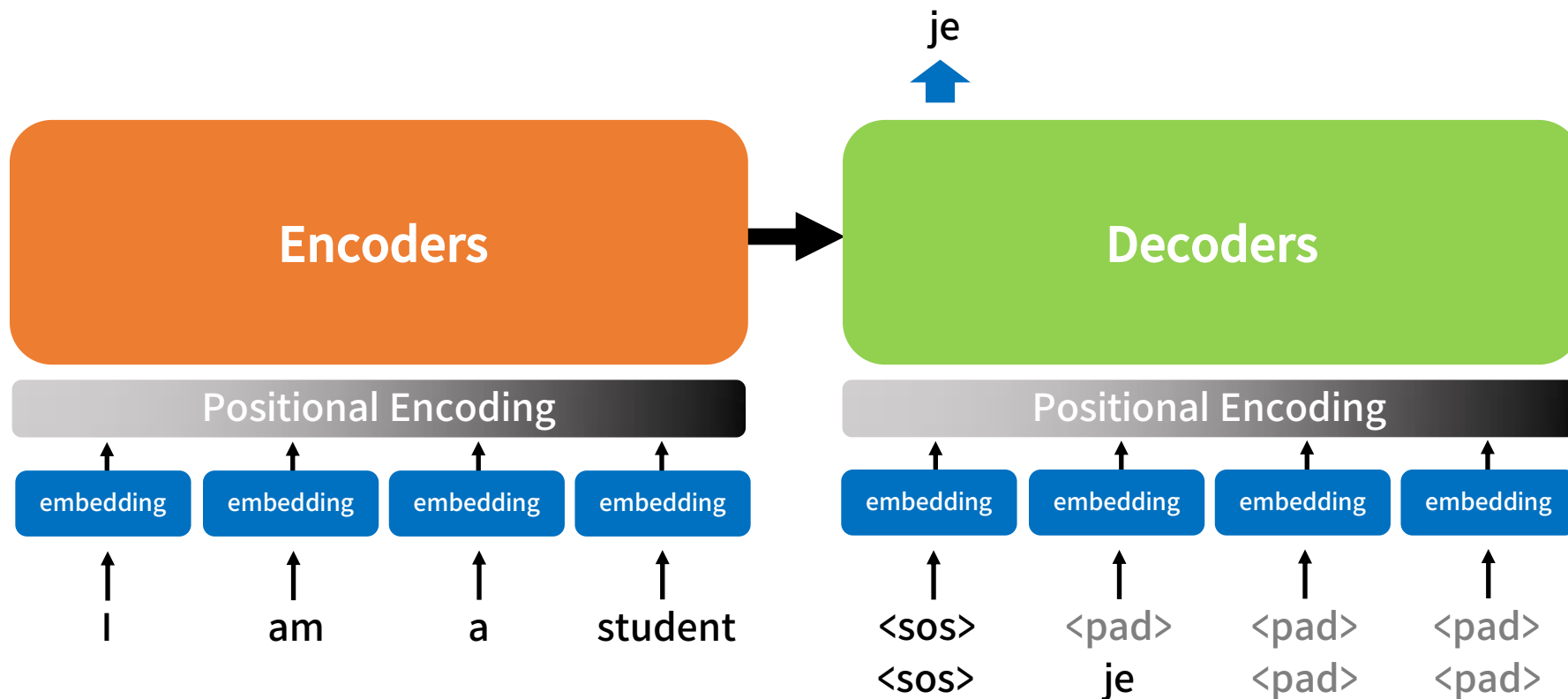
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



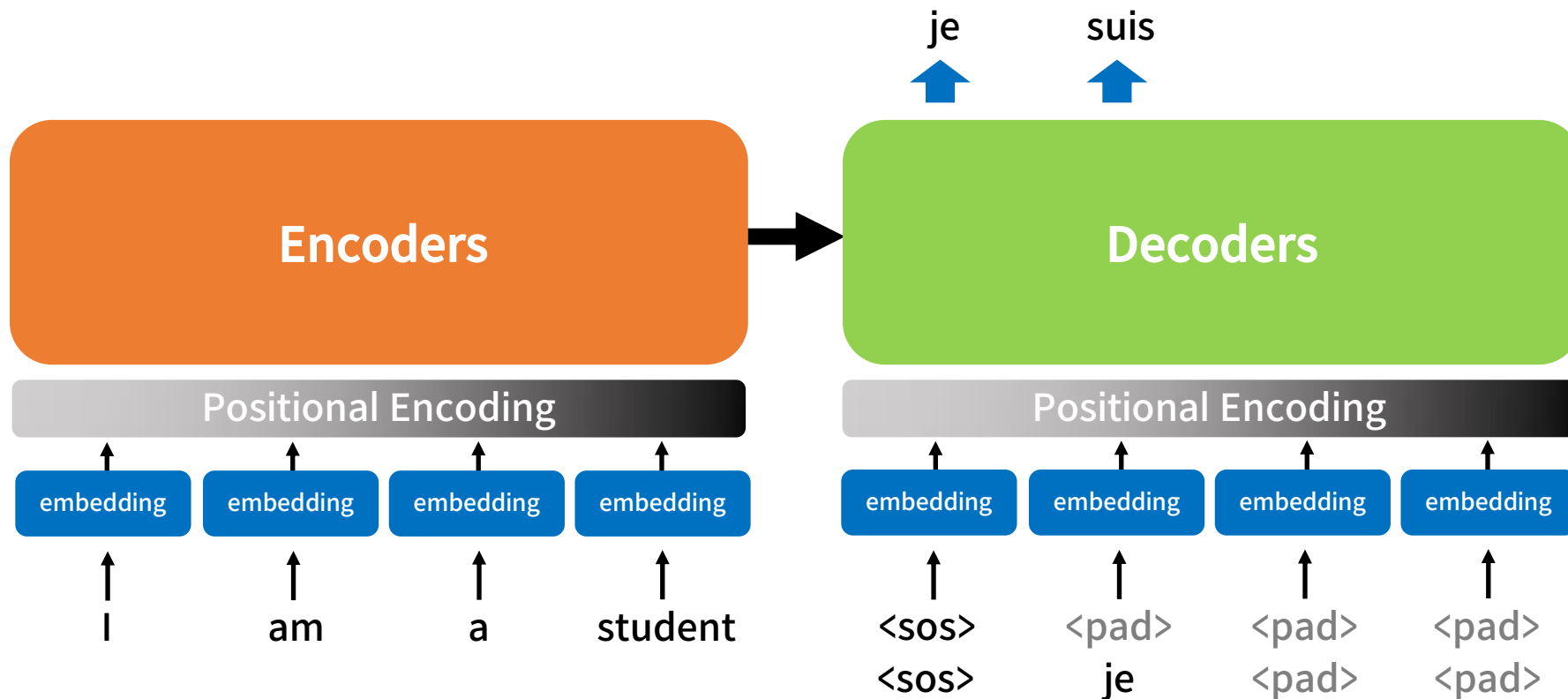
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



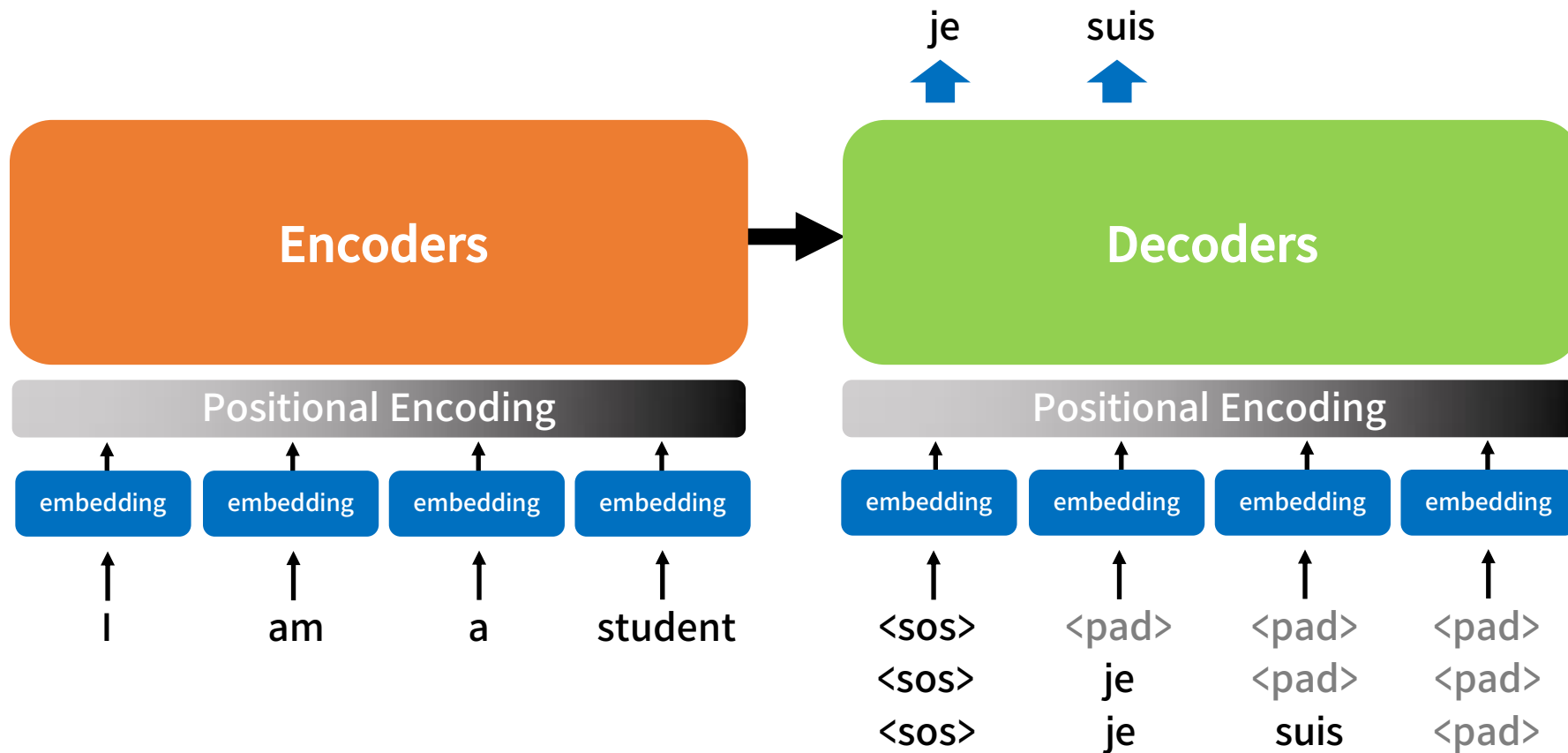
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



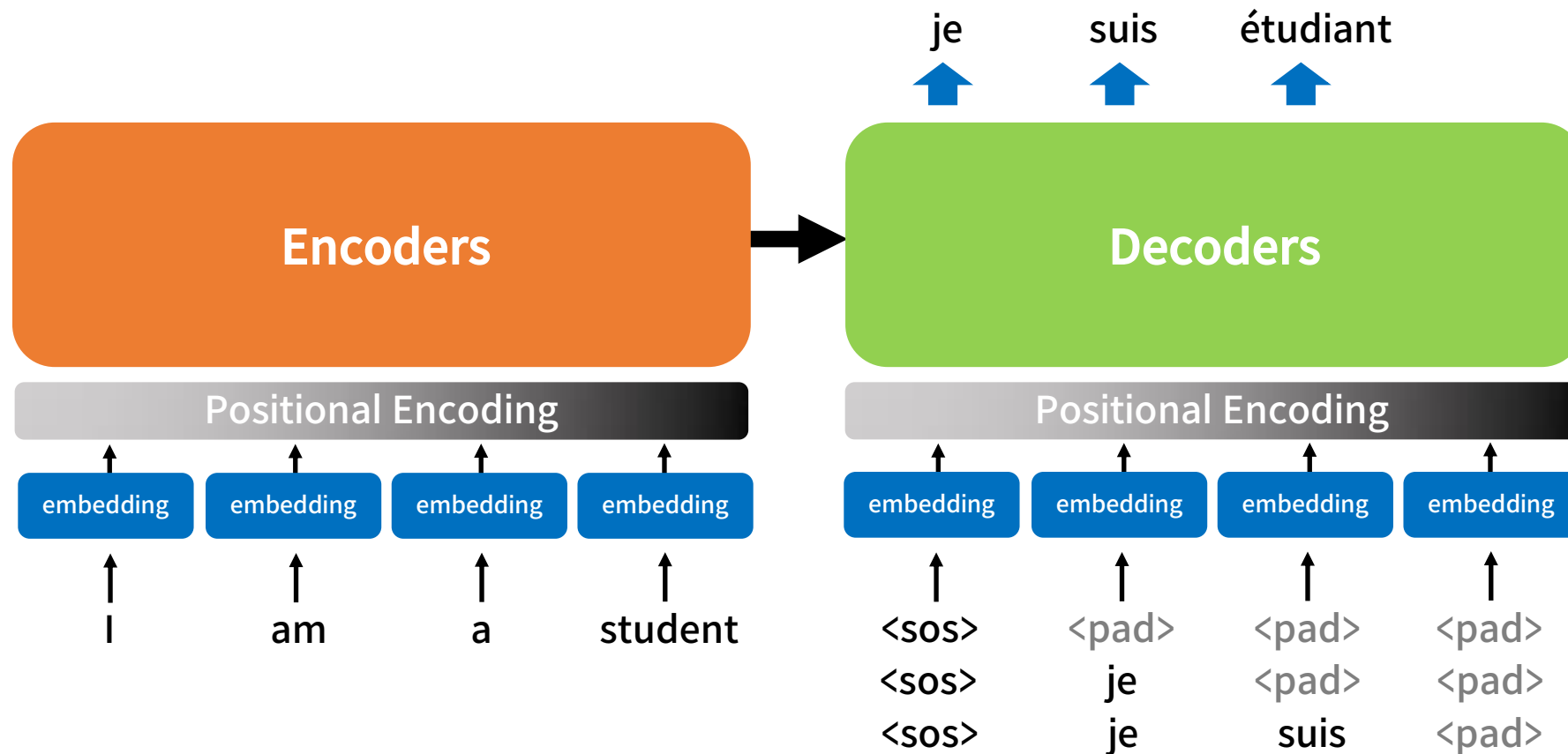
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



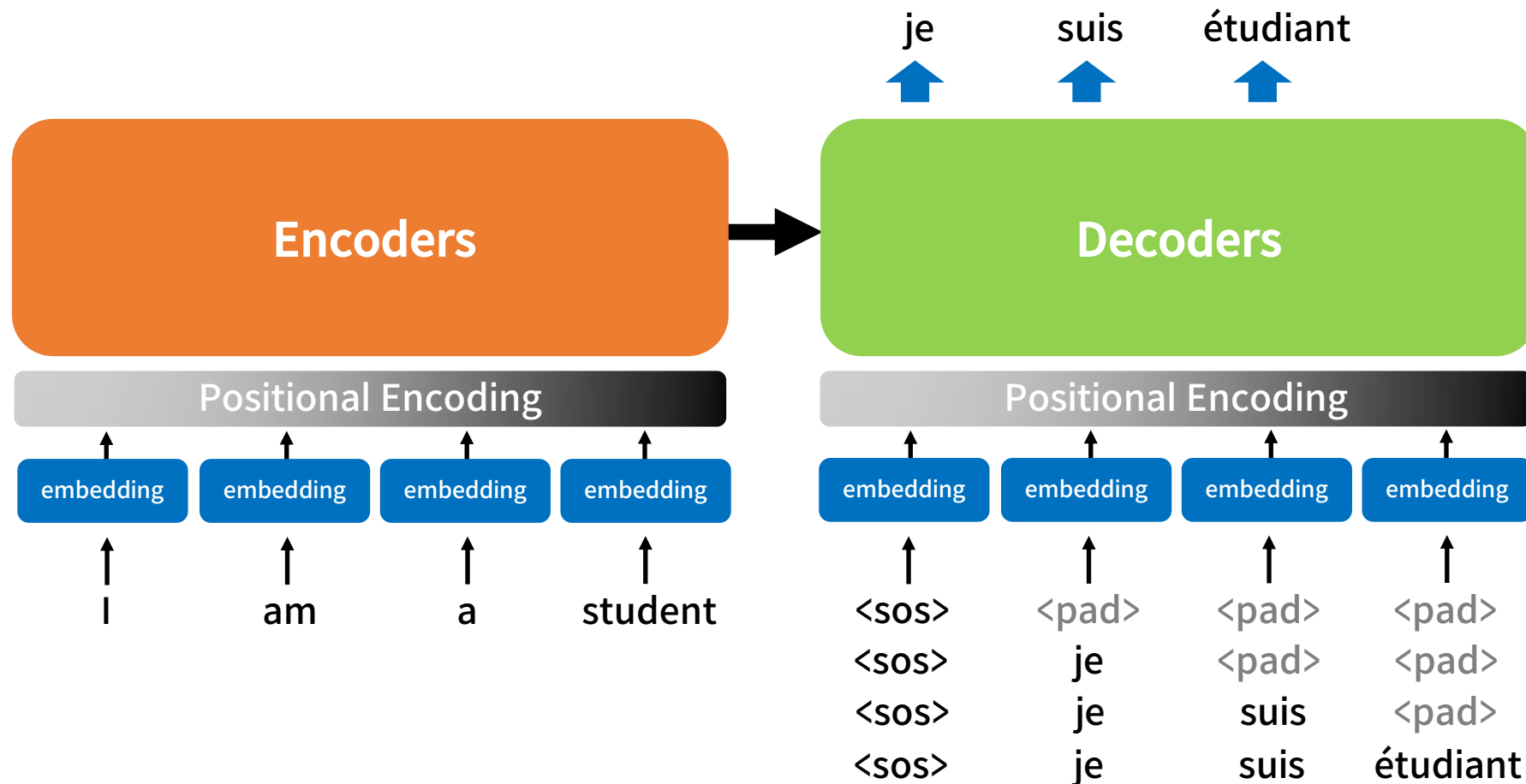
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



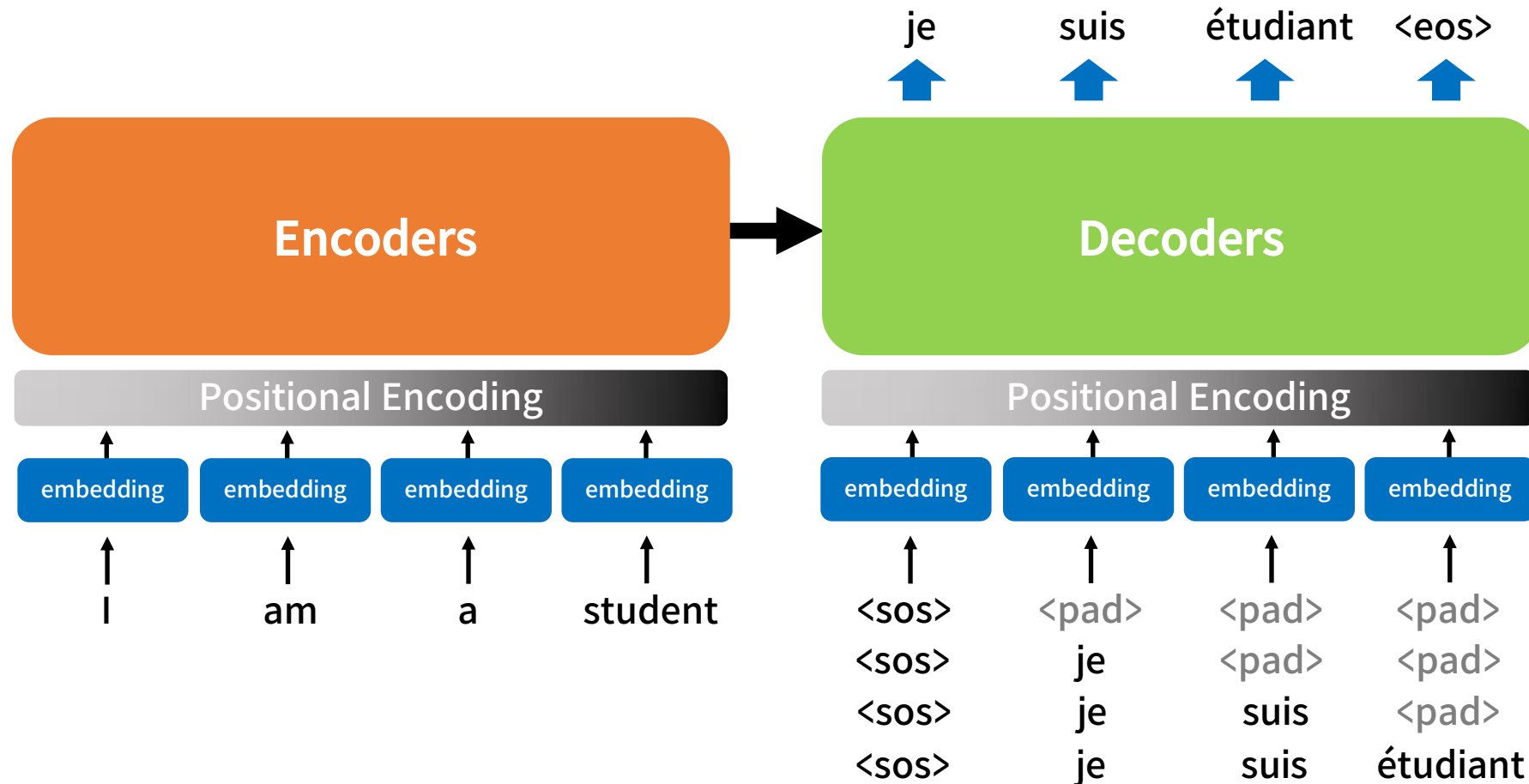
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



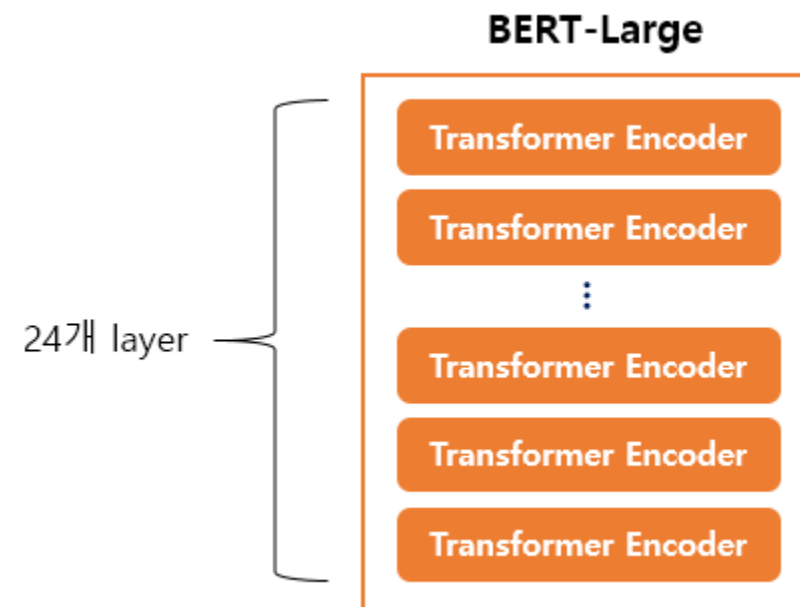
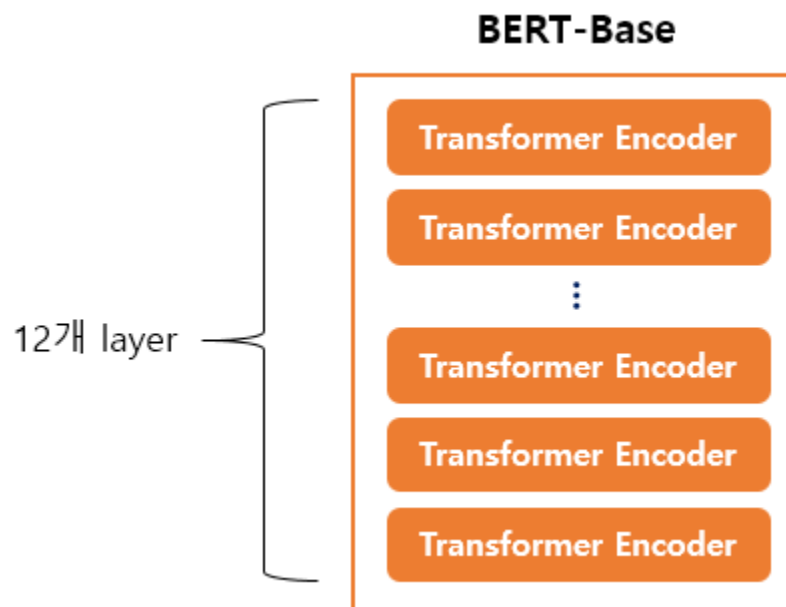
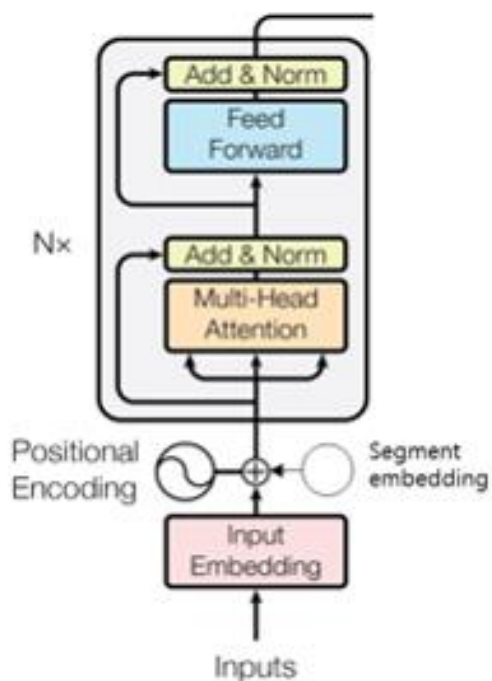
Transformer : Inference

- 테스트 단계(실제 번역기나 챗봇이 구동될 때)에는 디코더가 단어를 1개씩 생성해내야 한다.



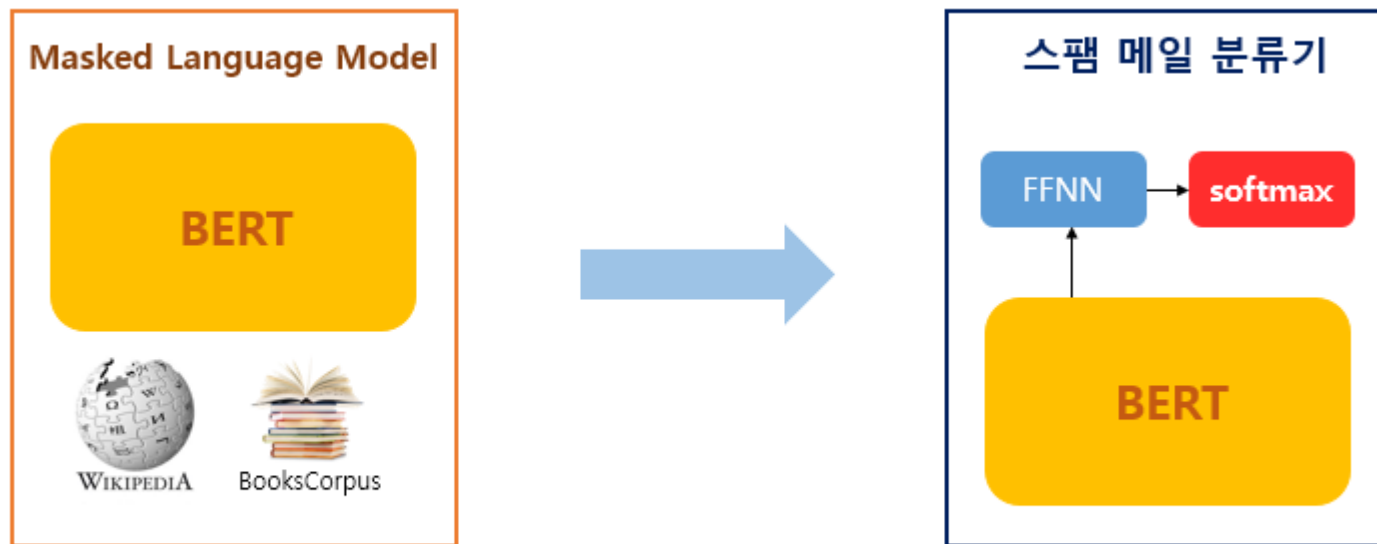
BERT-Base Vs. BERT-Large

- BERT-BASE는 트랜스포머의 인코더를 12층을 적재.
- BERT-LARGE는 트랜스포머의 인코더를 24층을 적재.



BERT의 적용

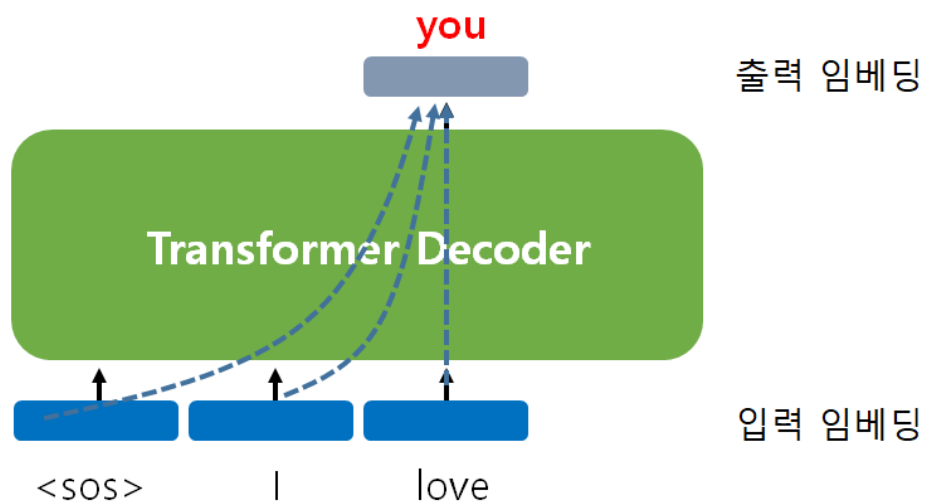
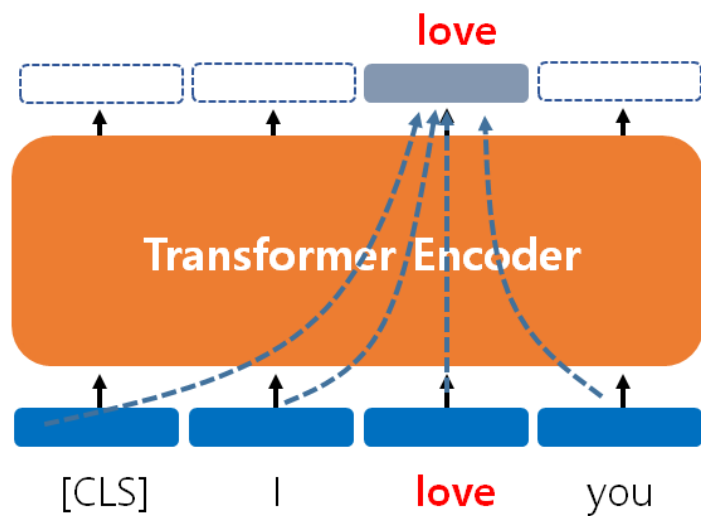
- 방대한 데이터로 사전 훈련된 언어 모델 BERT.
- BERT에 풀고자 하는 태크스에 맞는 추가적인 신경망을 추가.
- 풀고자 하는 태스크의 오차에 맞추어서 BERT를 학습. 이를 파인 튜닝(fine-tuning)이라 한다.



33억 단어에 대해서 4일간 학습시킨 언어 모델

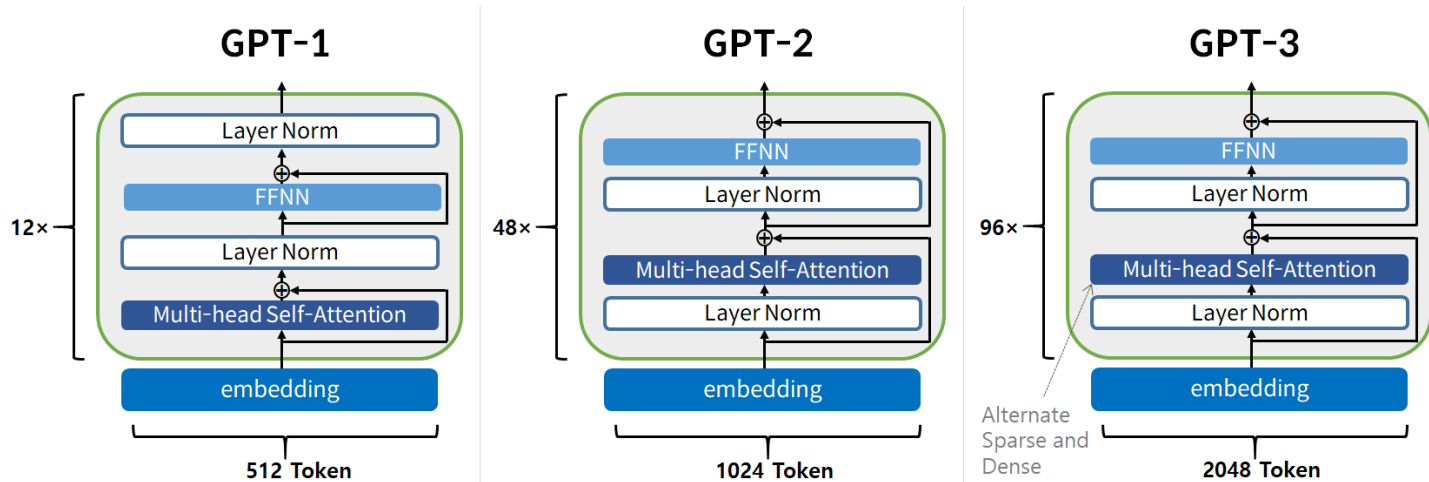
BERT Vs. GPT

- 양방향의 문맥을 반영하는 BERT. But NLG에 사용 불가.
- **이전 단어로부터 다음 단어를 예측하는 GPT.**



GPT의 발전

- GPT-1, GPT-2, GPT-3는 사실 아키텍처 면에서는 큰 차이가 없음.
- 차이는 학습한 데이터의 양과 모델의 크기.



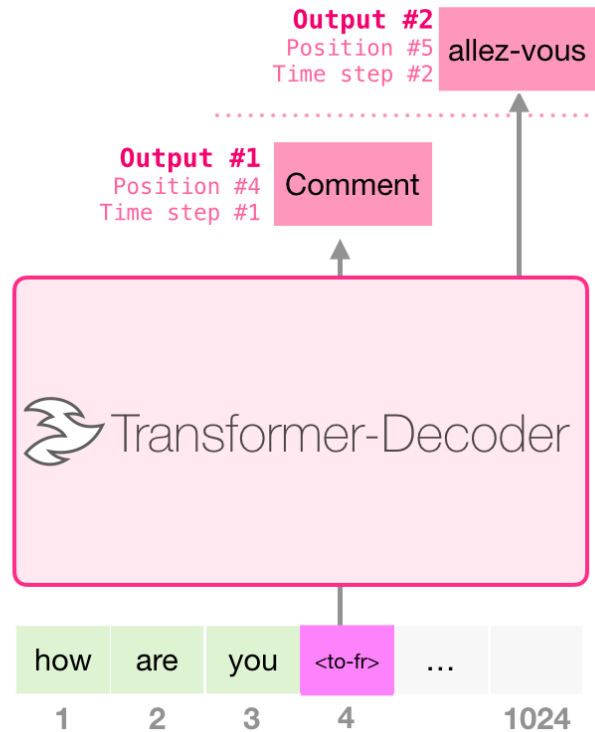
	GPT-1	GPT-2	GPT-3
파라미터 개수	1억 1,700만 개	15억 개	<u>1,750억 개</u>
디코더 블록을 쌓아올린 층 수	12개	48개	96개
입력 토큰 개수	512개	1024개	2048개

GPT를 이용한 기계 번역

- GPT의 사이즈가 크다면 다음과 같이 인코더 없이 번역 태스크를 학습하는 것도 가능.

Training Dataset

I	am	a	student	<to-fr>	je	suis	étudiant
let	them	eat	cake	<to-fr>	Qu'ils	mangent	de
good	morning	<to-fr>	Bonjour				



- 위키피디아 데이터를 이용하여 본문과 요약문의 학습셋을 구성하고 학습시킨다고 가정해보자.



WIKIPEDIA
The Free Encyclopedia

[Main page](#)
[Featured content](#)
[Current events](#)
[Random article](#)
[Donate to Wikipedia](#)
[Wikipedia store](#)

Interaction

[Help](#)
[About Wikipedia](#)
[Community portal](#)
[Recent changes](#)
[Contact page](#)

Tools

[What links here](#)
[Related changes](#)
[Upload file](#)
[Special pages](#)
[Permanent link](#)
[Page information](#)
[Wikipedia term](#)
[Cite this page](#)

Print/export

[Create a book](#)
[Download as PDF](#)
[Printable version](#)

Languages

[Español](#)
[Français](#)
[Italiano](#)
[日本語](#)
[Português](#)
[Svenska](#)
[Türkçe](#)
[Add 3 more](#)

[Edit this page](#)

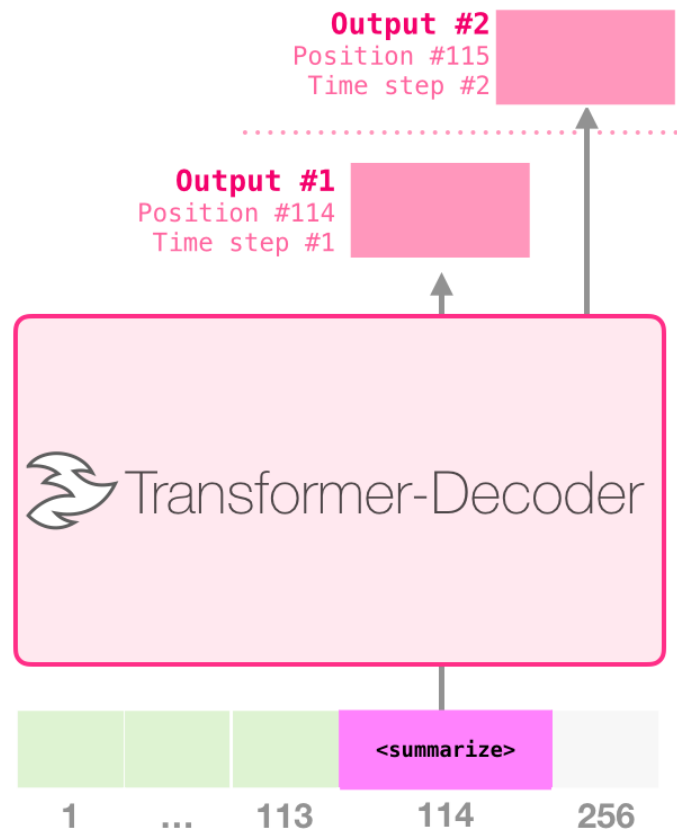
[illegible]

GPT를 이용한 텍스트 요약

- 위키피디아 데이터를 이용하여 본문과 요약문의 학습셋을 구성하고 학습시킨다고 가정해보자.

Training Dataset

Article #1 tokens	<summarize>	Article #1 Summary	
Article #2 tokens	<summarize>	Article #2 Summary	padding
Article #3 tokens	<summarize>	Article #3 Summary	



BART와 T5

BART는 양방향(bidirectional) 및 자기회귀(auto-regressive) 모델로서, 인코더(encoder)와 디코더(decoder)를 모두 사용함
인코더는 입력된 텍스트를 인코딩하여 고정 길이 벡터로 만들고, 디코더는 이 벡터를 이용하여 출력 텍스트를 생성

텍스트 분류

카테고리 3개 : [0, 1, 2]

모델을 학습시키고 나서 분류를 한다고 하면
입력 텍스트 >> 모델 >> 0, 1, 2 예측

=====

카테고리 (긍정, 중립, 부정)

입력 텍스트 >> 모델 >> '긍정', '중립', '부정' 이라는 단어 생성

카테고리 : 긍정, 중립, 부정 중에서 무조건 생성해야 한다(조건 강제)

데이터:

주식회사 한시경의 매출이 급증했다 >>> 긍정

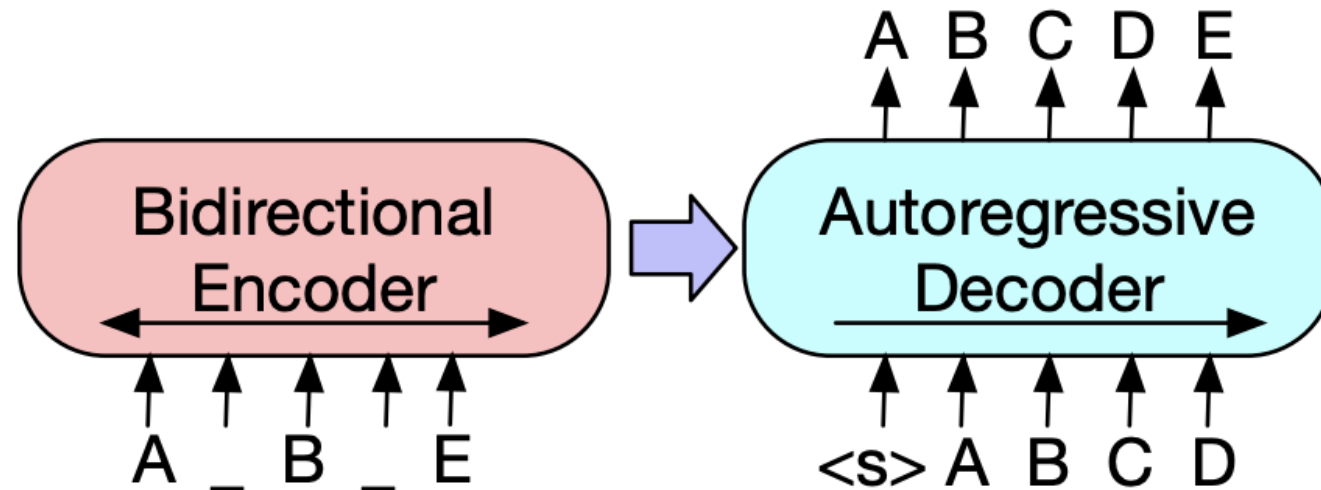
ChatGPT 등장으로 스타트업에게는 기회일 수도 있고 위협일 수도 있다 >>> 중립

=====

삼성전자가 1분기 300억 적자가 났다 >>> 부정

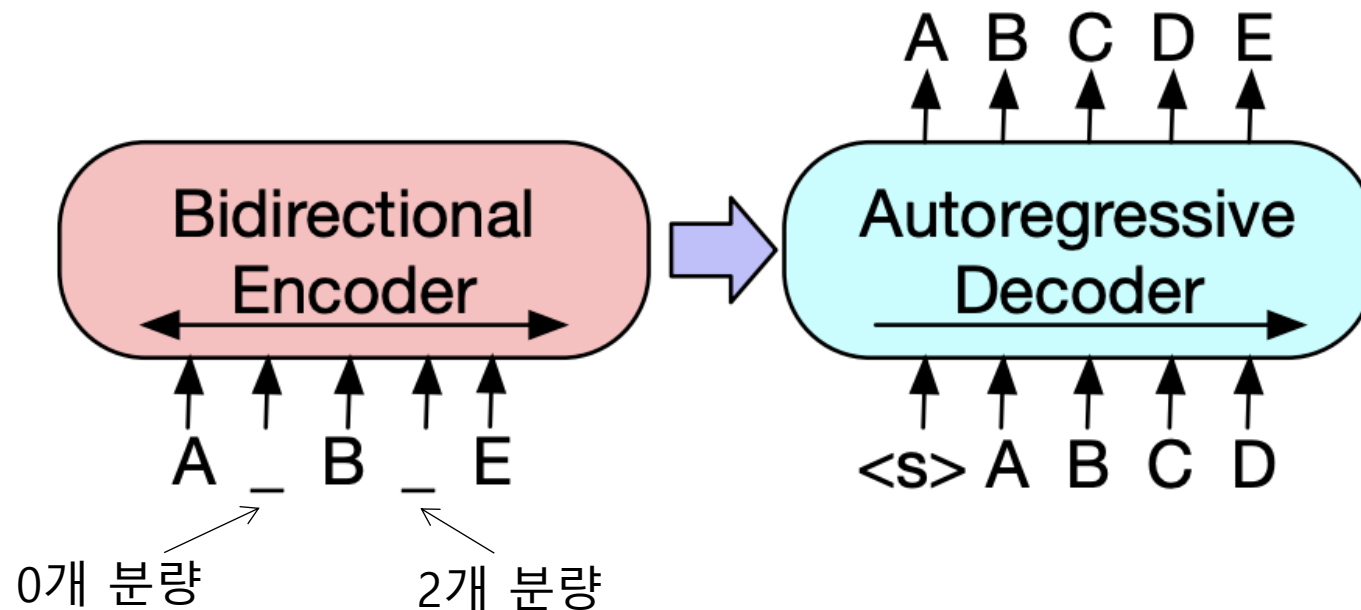
BART, Bidirectional Auto-Regressive Transformer

- 트랜스포머 인코더 => MLM으로 가운데 단어 예측 => BERT => NLU에 강하다.
- 트랜스포머 디코더 => 이전 단어들로부터 다음 단어 예측 => GPT => NLG에 강하다.
- 기존의 트랜스포머는 인코더-디코더 구조였다. 그렇다면 인코더-디코더 구조 자체를 다시 활용해보자.



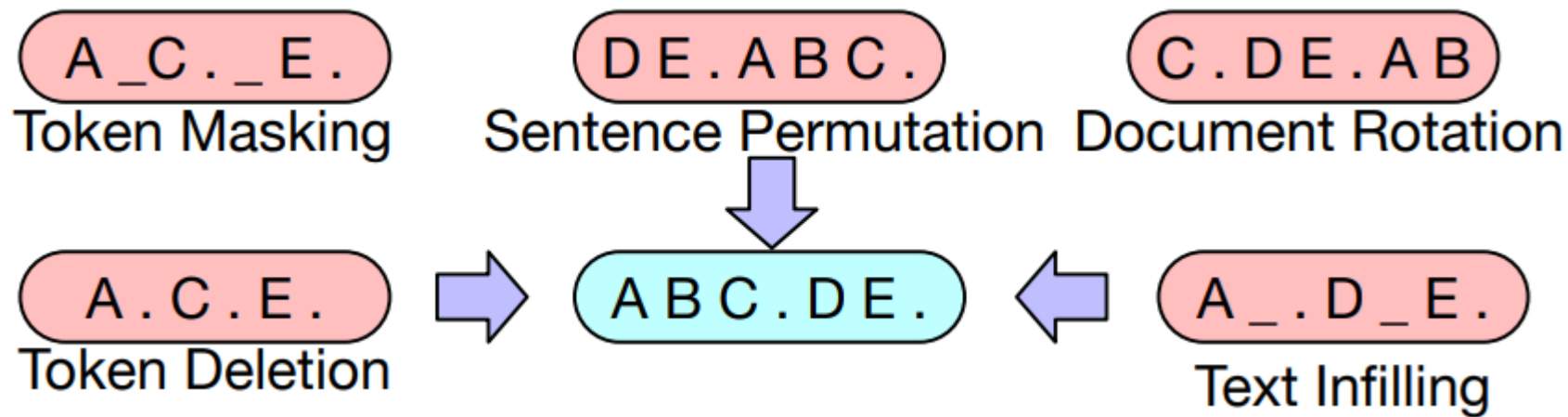
BART, Bidirectional Auto-Regressive Transformer

- 트랜스포머 인코더-디코더를 모두 사용하여 Pre-training
- 입력 문장에 노이즈 추가(Masking)
- 인코더로부터 정보를 받아 디코더에서 원래 문장을 복원.
- 인코더-디코더 구조는 입, 출력의 길이가 달라도 되므로 유연한 Masking이 가능.



BART의 Pre-training

- 다양한 Noise를 추가하는 방식을 사용함.
- BERT에 비해 다양한 Pre-training으로 NLU 성능도 Up.
- 그리고 BERT와 달리 디코더도 있으므로 NLG 또한 가능.



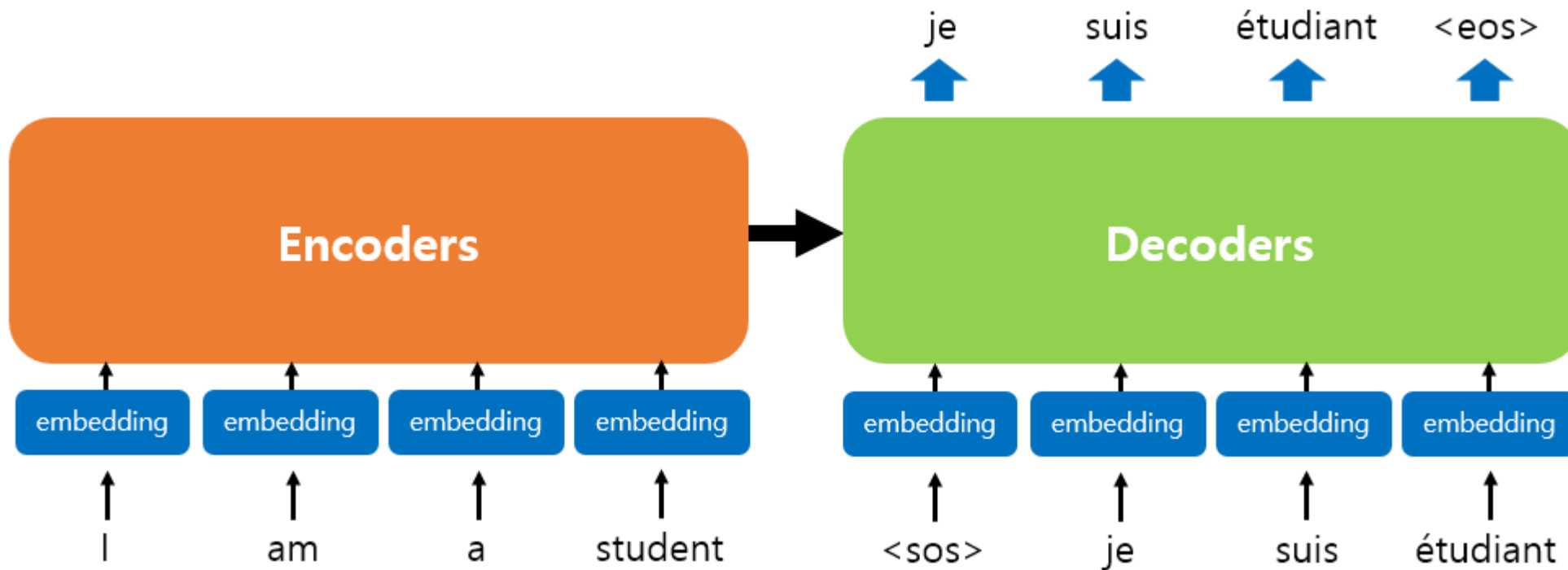
BART의 NLU 성능

- NLU 태스크에서도 강한 모습을 보임.
- BERT는 당연히 이기고, RoBERTa(BERT의 개선 버전)과 유사한 성능.

	Question Answering	Question Answering	Textual Entailment	Text Classification	Text Comparison	Question Answering	Text Comparison	Textual Entailment	Text Classification	Linguistic Acceptability
	SQuAD 1.1 EM/F1	SQuAD 2.0 EM/F1	MNLI m/mm	SST Acc	QQP Acc	QNLI Acc	STS-B Acc	RTE Acc	MRPC Acc	CoLA Mcc
BERT	84.1/90.9	79.0/81.8	86.6/-	93.2	91.3	92.3	90.0	70.4	88.0	60.6
UniLM	-/-	80.5/83.4	87.0/85.9	94.5	-	92.7	-	70.9	-	61.1
XLNet	89.0 /94.5	86.1/88.8	89.8/-	95.6	91.8	93.9	91.8	83.8	89.2	63.6
RoBERTa	88.9/ 94.6	86.5 / 89.4	90.2 / 90.2	96.4	92.2	94.7	92.4	86.6	90.9	68.0
BART	88.8/ 94.6	86.1/89.2	89.9/90.1	96.6	92.5	94.9	91.2	87.0	90.4	62.8

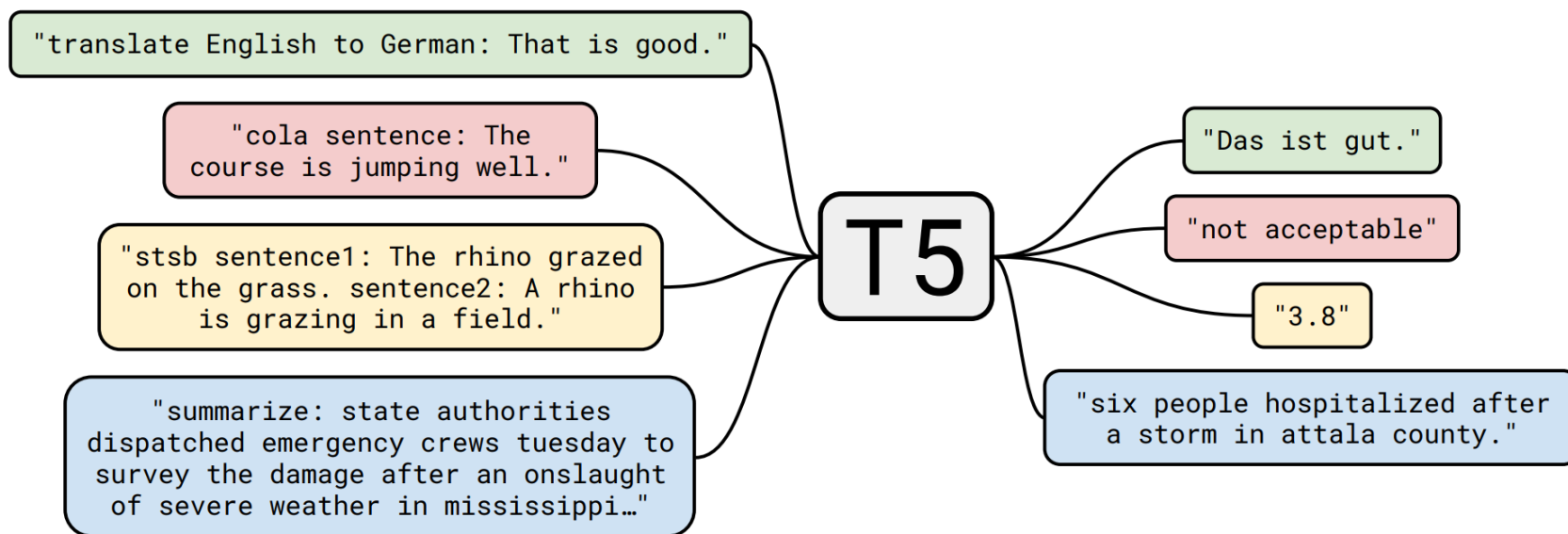
BART를 이용한 텍스트 요약

- 번역기를 위해 탄생하였던 트랜스포머와 다를 바 없음.
- 인코더 디코더 구조로 이루어진다.
- 인코더의 입력, 디코더의 입력, 디코더의 레이블로부터 학습 후 텍스트를 요약함.



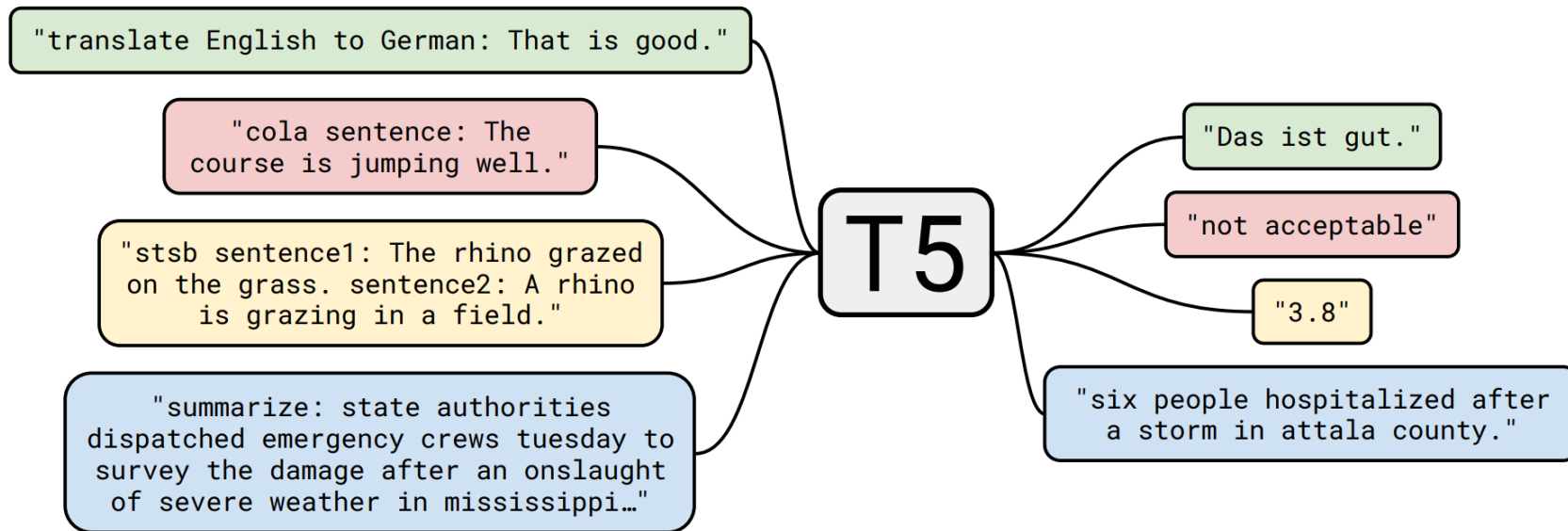
Text-to-Text Framework

- Google T5에서는 새로운 Fine-tuning 방식을 사용.
- BART와 같이 enc-dec 방식.
- 모든 downstream task들을 하나의 방식(NLG)로 처리 => 텍스트 분류에서도 클래스 인덱스 예측 방식 No.
- 별도의 새로운 layer 추가 필요없이 기존 모델 그대로 fine-tuning 가능



Text-to-Text Framework

- 모든 downstream task들을 하나의 방식(NLG)로 처리 => 텍스트 분류에서도 클래스 인덱스 예측 방식 No.
- 그림에서 빨간색과 노란색은 원래 텍스트 분류(NLU) 문제
- 참고 영상 : https://www.youtube.com/watch?v=upL76wu1EVQ&ab_channel=naver2



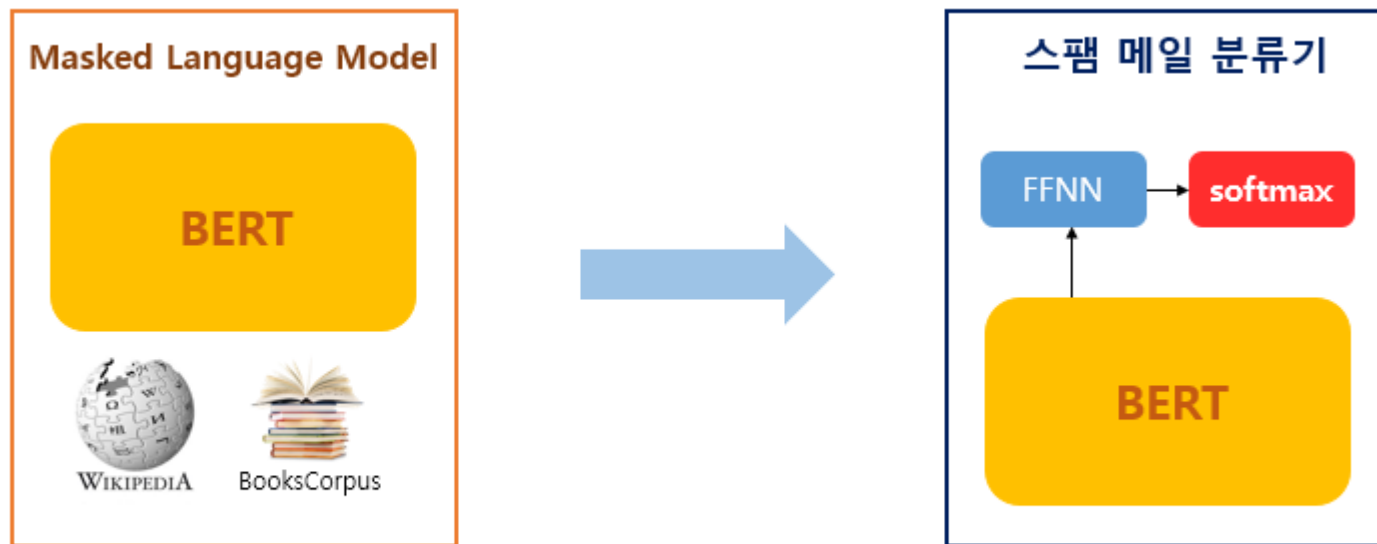
GPT-3와 LLMs

GPT에서 Downstream Task란?

GPT 모델이 전체 문장을 이해하고 생성하는 능력을 바탕으로 특정 자연어 처리 태스크를 수행하기 위해 Fine-tuning을 거쳐 만들어지는 모델을 말함

Fine-tuning의 필요성?

- 대부분의 PLM은 사전 학습 단계 이후에 fine-tuning을 통해 Task를 학습함.
- 이러한 fine-tuning은 수천~수만의 데이터셋을 필요할 뿐만 아니라 오버피팅되기 쉬움.
- 각 Downstream Task마다 fine-tuning된 모델들을 각각 만들어야함.
- 사람은 예제 몇 개만 주면 바로 이해하고 문제를 풀 수 있다는 것과 대조됨.



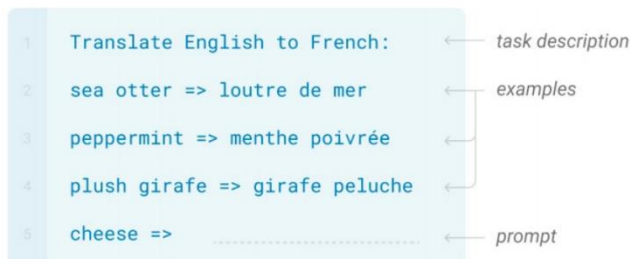
33억 단어에 대해서 4일간 학습시킨 언어 모델

In-context Learning

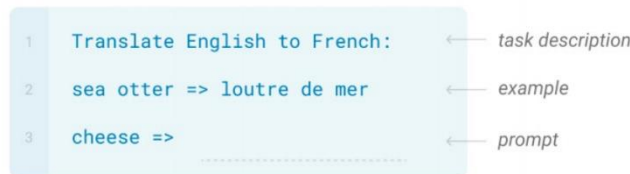
- GPT-3가 별도의 fine-tuning 과정없이 예시만으로 학습한 효과를 낼 수 있는 것.
- LLM에게 예시 없이(zero-shot), 예시 1개(one-shot), 예시 몇 개(few-shot)으로 문제를 풀 수 있게 해보자.
- 프롬프트 엔지니어링의 시작.

- 모두 가중치 파라미터 업데이트는 없음

- Few-shot



- One-shot



- Zero-shot



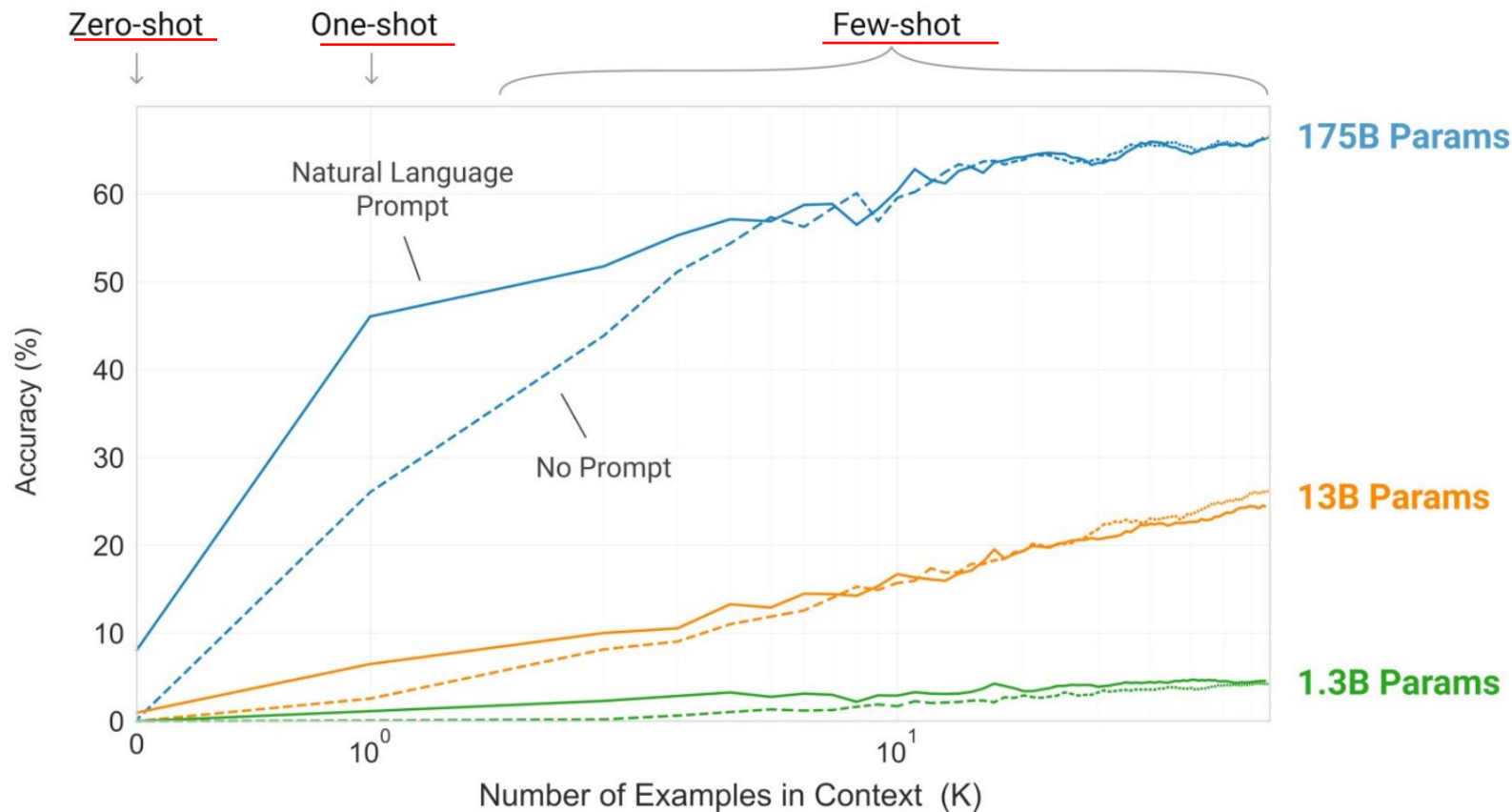
+Fine-tuning



In-context Learning

- 모델이 커질수록 In-context Learning의 효율성이 높아짐.

문맥 안에서 배움



현재의 LLM Q&A

- 일반적으로 LLM을 (10B, 100억개의 파라미터)를 가진 모델이라고 정의.
- BERT, RoBERTa, GPT-2, BART, T5 등은 해당되지 않음.
- 현재의 LLM은 대부분 GPT 계열의 디코더 모델이 대부분.

질문 1)

- 10B 이상의 모델을 서비스에 적용하는 것이 쉬운가?
- 텍스트 요약, 번역기 등에 사용할 수는 있겠지만 현실적으로 인퍼런스 비용 문제가 있음.

질문2)

- 10B 이상의 모델이 오픈소스로 공개되어 있는가?
- Polyglot-ko 12.8B와 같이 공개는 되어져 있음.

질문3)

- 오픈소스로 공개된 10B 내외의 모델이 ChatGPT를 대체가능할까? Ex) KoAlpaca 등...
- ChatGPT의 파라미터를 175B라고 추정한다고 하였을 때, 절대 성능적으로 따라잡을 수 없음.
- 그나마 흉내내려면 최소 30B, 60B는 되어야함.

LLM의 튜닝 방법 (LoRA 학습)

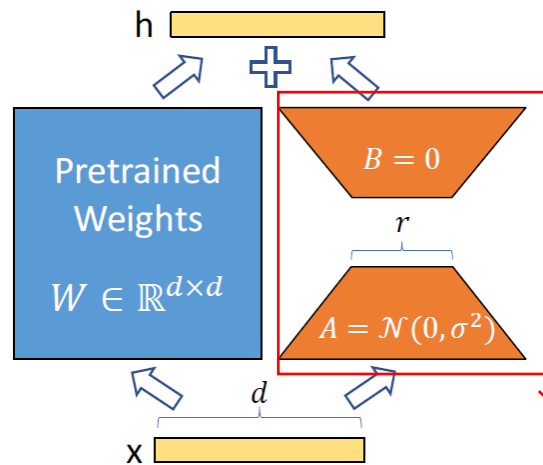
- 일반적으로 LLM을 (10B, 100억개의 파라미터)를 가진 모델이라고 정의.
- 10B 이상의 LLM을 fine-tuning하기란 쉽지 않음.
- 이를 위해 대표적으로 사용되는 방법이 LoRA 학습.
- 사전훈련된 가중치는 그대로 두고, 거기에 별도로 추가된 레이어만 새로운 데이터로 학습.

- 링크 : <https://github.com/Beomi/KoAlpaca>

- 2023.03.26: 🤗LLAMA 30B 기반 KoAlpaca 모델을 공개합니다. (LoRA로 학습)
 - LLAMA 30B 학습은 GIST Sundong Kim 교수님의 A100 지원으로 학습되었습니다. 감사합니다 🙏
- 2023.03.24: 🤗LLAMA 13B 기반 KoAlpaca 모델을 공개합니다. (LoRA로 학습)
- 2023.03.23: 🤗LLAMA 65B 기반 KoAlpaca 모델을 공개합니다. (LoRA로 학습)
- 2023.03.22: 카카오톡에 포팅된 KoAlpaca 봇이 추가되었습니다.

LoRA tuning

- GPT-3같은 거대한 모델을 fine-tuning하면 그 엄청난 parameter들을 다 재학습시켜야 함
- Transformer의 각 layer마다 학습가능한 행렬을 별도로 추가하는 방식. (기존의 파라미터는 고정)
- 레이어 중간중간마다 존재하는 hidden states h 에 값을 더해줄수 있는 파라미터를 추가 추가 층(layer) 주는 방식
- PLM의 파라미터는 건드리지 않으므로 여러 버전의 LoRA를 만드는 것이 가능함.



$$h = W_0 x + B A x$$

Figure 1: Our reparametrization. We only train A and B .



박스 부분만 훈련시킴

NLP 엔지니어로서의 방향성

- 대부분의 기업에서 여전히 LLM을 사용할 수 없는 상황.
➔ BERT, RoBERTa, GPT-2, BART, T5를 사용할 수 있는 것이 여전히 아직은 경쟁력임.
- 예를 들어 텍스트 분류, NER, MRC 문제는 BERT, RoBERTa, T5가 여전히 강력함.
- LLM은 사용할 수 없어도 ChatGPT API, GPT-4 API는 사용할 수 있음.
➔ ChatGPT API를 이용한 데이터 레이블링 작업 후 위에서 언급한 모델을 튜닝.
➔ ChatGPT API, Embedding API를 사용하여 사내 데이터를 이용한 챗봇 개발.
➔ langchain, AutoGPT 등을 이용한 어플리케이션 개발.